

```

6.  if return_value > 0:
7.      gain_days += 1
8.  else:
9.      loss_days += 1
10.
11. win_rate = gain_days / len(daily_returns) * 100
12. print(f"Win rate: {win_rate}%")
13.

```

This loop calculates a simple win rate by counting positive and negative return days—a basic but important metric for evaluating trading strategies.

Nested Loops: Analysis Across Multiple Dimensions

Sometimes we need to loop through multiple dimensions, like analyzing each stock across multiple timeframes. Nested loops (loops within loops) handle this:

```

1. timeframes = ["daily", "weekly", "monthly"]
2. stocks = ["AAPL", "GOOGL", "AMZN"]
3.
4. for stock in stocks:
5.     for timeframe in timeframes:
6.         print(f"Analyzing {stock} on {timeframe} timeframe")
7.
8. This code cycles through each combination of stock and timeframe, giving us 9 combinations in total.
9. Here's a more sophisticated trading example:
10. sectors = ["Technology", "Healthcare", "Finance"]
11. metrics = ["P/E Ratio", "Debt/Equity", "Profit Margin"]
12.
13. for sector in sectors:
14.     print(f"\nAnalyzing {sector} sector:")
15.     for metric in metrics:
16.         print(f" - Calculating average {metric}")
17.

```

This nested loop structure could form the foundation of a sector rotation strategy, where a trader analyzes multiple financial metrics across different